



Python Developer - Home Assignment Report

~By: Shubham Wagh
(shrishubhamwagh@gmail.com)

Date: 06-08-2025

Table of Content:

1. Introduction.....	2
1.1 Objective.....	2
1.2 Tools used.....	2
1.3 Basic Data Flow.....	3
2. Methodology.....	4
2.1 Data Collection from Reddit.....	4
2.2 Sentiment Analysis.....	4
2.3 Data Enrichment.....	5
2.4 Heuristic Score Development Using Gemini LLM.....	5
2.5 Saving Results.....	6
3. Results.....	7
3.1 Data Overview.....	7
3.2 LLM-Based Scoring Outcomes.....	8
3.3 Observations & Insights.....	9
4. Discussion.....	10
4.1 Conclusion.....	10
4.2 Future Scope.....	10

1. Introduction

In this digitally inclined generation, the internet gives everyone a voice, a freedom of speech beyond national boundaries. But sometimes, that voice can be used to spread hate. In recent years, platforms like Reddit have seen a rise in harmful and antisemitic content, often hidden within threads, comments, and communities. Spotting and tracking this kind of content manually is nearly impossible at scale. That's where automation and data analysis come in, and through this ActiveFence assessment we are trying to make it happen.

The goal of this project is to build a Python-based system that can help identify antisemitic behaviour on Reddit. The result is a working system that can help track antisemitic content on Reddit, with clear explanations for each score and a structure that can be scaled or adapted for future needs. Moreover, it leverages the LLM capabilities to rank the radicalised content/user along with the explanation behind its decision.

1.1 Objective

This report outlines the end-to-end process of building a pipeline that collects, enriches, and analyses Reddit data. The project is designed to meet the following key objectives:

- Automatically collect Reddit posts discussing controversial, harmful, or antisemitic topics.
- Score posts based on their likelihood of containing antisemitic content, with a focus on explainability.
- Analyse user behaviour over time and assign a user-level radicalisation score.
- Handle edge cases such as new or private users.
- Provide a framework for daily monitoring and alerting for high-risk users.

1.2 Tools used

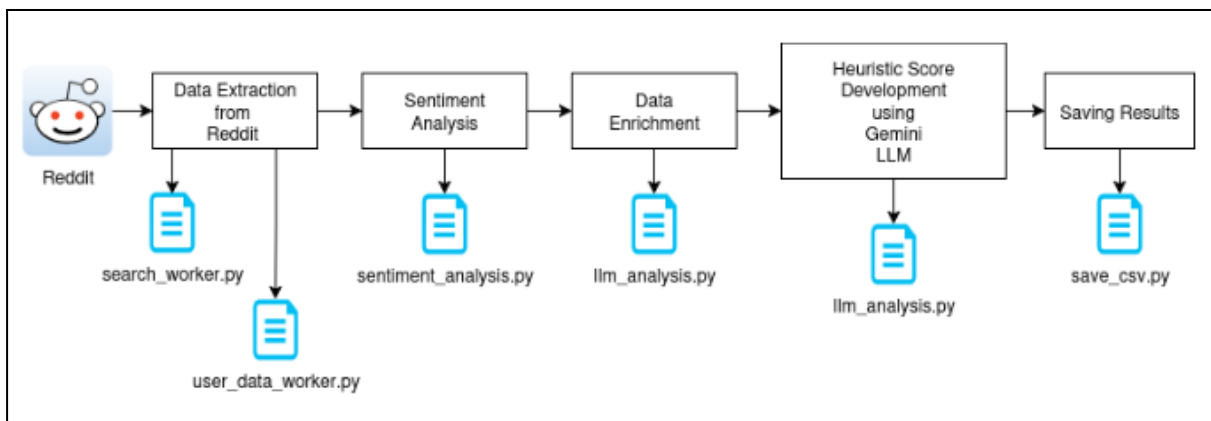
Throughout the project, there are certain tools and python libraries used, as mentioned below:

Category	Tool / Technology	Purpose
Programming Language	Python	Core language for scraping, processing, and analysis
Code Repository	GitHub	Version control and code hosting
Data Formats	JSON, CSV	Storing collected data and analysis results
LLM Integration	Gemini 2.5 (Flask API on Google Cloud)	Live large language model for post scoring and explanation generation

Development Environment	Visual Studio Code (VSCode)	Writing, testing, and debugging the code
Libraries & Dependencies	<code>praw</code>	Reddit API access and data scraping
	<code>pandas</code>	Data manipulation and structuring
	<code>nltk</code>	Natural language processing tools
	<code>requests</code>	Making API calls (e.g., to LLM endpoint)
	<code>python-dotenv</code>	Managing API keys and environment variables securely
	<code>pytest</code>	Testing the code for reliability
	<code>loguru</code>	Structured logging and debugging
	<code>ratelimit</code>	Handling API rate limits gracefully

1.3 Basic Data Flow

Below is the basic data flow block diagram, which tells us exactly how the system is processing the data and delivering the outcome.



2. Methodology

This section describes the complete process followed to collect, analyze, and score Reddit content for potential antisemitic behavior. The system was implemented in **Python** and leverages **Gemini-2.5-flask LLM model**, also the system is designed to be modular, scalable, and easy to maintain.

2.1 Data Collection from Reddit

The process begins with collecting Reddit posts related to potentially controversial or harmful topics, with a particular focus on antisemitism. A set of predefined search terms and relevant subreddits were configured to guide the scraping.

- The system uses a list of **predefined keywords** ["jew", "zionist", "holocaust denial", "antisemitic"] and **target subreddits** ["conspiracy", "politics"] (configured in `config.py`)
- A total of **160 posts** (20 posts x 4 keywords x 2 subreddits) were initially collected across target subreddits.

```
def search_and_collect(reddit, subreddit, term, limit=20):
    local_results = []
    logger.info(f"Searching '{term}' in r/{subreddit}")
    try:
        for submission in reddit.subreddit(subreddit).search(term, sort='new', limit=limit):
            local_results.append({
                'id': submission.id,
                'subreddit': str(submission.subreddit),
                'title': submission.title,
                'selftext': submission.selftext,
                'author': str(submission.author) if submission.author else '[deleted]',
                'created_utc': datetime.utcfromtimestamp(submission.created_utc).isoformat(),
                'score': submission.score,
                'num_comments': submission.num_comments,
                'url': submission.url
            })
    except Exception as e:
        logger.info(f"Error searching '{term}' in r/{subreddit}: {e}")
    return local_results
```

Reddit's official API was accessed using the `praw` library, and all posts were stored in structured CSV format (`harmful_posts.csv`).

2.2 Sentiment Analysis

Once potentially harmful posts were identified, a sentiment analysis step was applied using the **NLTK's VADER** sentiment analyzer library.

- Each post was classified as **Problematic**, **Non-problematic**, and **Neutral**, based on compound attributes given by NLTK VADER.
- From 160 posts collected earlier, nearabout **48 posts** were flagged as potentially problematic and added to `problematic_users.csv`.
- Each flagged post included metadata like post content, author, date, and subreddit.

- This helped filter out false positives and provided additional context to prioritize users for further review.

```
def analyze_sentiment(csv_file):
    nltk.download('vader_lexicon')
    df = pd.read_csv(csv_file)
    titles = df['title']
    sia = SIA()
    results = []

    for line in titles:
        pol_score = sia.polarity_scores(line)
        pol_score['title'] = line
        results.append(pol_score)

    df_results = pd.DataFrame(results)
    # Add statement column based on thresholds
    df_results['statement'] = 'Neutral'
    df_results.loc[df_results['compound'] > 0.2, 'statement'] = 'Non-problematic'
    df_results.loc[df_results['compound'] < -0.2, 'statement'] = 'Problematic'
    df_final = pd.concat(
        [df.reset_index(drop=True), df_results[['compound', 'statement']].reset_index(drop=True)],
        axis=1
    )
    return df_final
```

Users linked to problematic or concerning content were flagged for deeper analysis in the next steps.

2.3 Data Enrichment

For each user flagged during sentiment analysis, their recent activity was fetched to understand behavioral patterns over time.

- The system collected up to 2 months (60 Days) of Reddit activity per user, including both posts and comments.
- A total of around 2431 items (posts, comments, or fetch errors) were gathered and saved to `user_data_last_2_months.csv`.
- Data was also converted to JSON format (`enriched_data.json`) to support flexible downstream processing.

This enrichment step provided valuable historical context and made it possible to evaluate users holistically rather than based on a single post.

2.4 Heuristic Score Development Using Gemini LLM

To evaluate the risk level of users more precisely, we integrated and used a live LLM (Gemini 2.5) using GEMINI_ENDPOINT and GEMINI_API_KEY.

- The model was queried with each user's recent activity like recent posts & comments to assess the likelihood of antisemitic or radicalised behaviour.
- Gemini LLM returned both a score and a descriptive explanation for each user.
- Results were saved to `llm_analysis_results.csv`, containing both the score and the reasoning behind it.

```

@sleep_and_retry
@limits(calls=60, period=60)
def query_gemini(username, activity_list):
    prompt = f"""
    You are an expert in hate speech detection and social media analysis.

    You will be given a list of Reddit post titles made by a user over the past 2 months. Your task is to analyze the content for signs of anti-Semitism.

    Rate the user's activity with:
    - A score from 0 (no anti-Semitism) to 10 (severe anti-Semitic behavior)
    - A brief explanation justifying your score (2-4 sentences)

    IMPORTANT: Return ONLY the JSON output. Do not include any explanations, greetings, or markdown formatting like ```json.

    The output must be valid JSON in this format:
    {{
      "username": "{username}",
      "score": SCORE_HERE,
      "explanation": "Your explanation here"
    }}

    Reddit post titles for user "{username}":
    {json.dumps(activity_list, indent=2)}
    """

    headers = {
        "Content-Type": "application/json",
        "X-goog-api-key": GOOGLE_API_KEY
    }

    payload = {
        "contents": [
            {"parts": [{"text": prompt}]}
        ]
    }

```

- Currently, we are using the free version of Gemini-2.5-Flask, so rate limiting is the factor, but in order to bypass that, we are using the ratelimit library, which is firing 60 requests per 60 seconds, bypassing the rate-limiting stuff.

This approach brought explainability to the scoring and allowed more nuanced assessments than simple keyword matching.

2.5 Saving Results

All intermediate and final outputs were saved in structured formats:

File Name	Description
harmful_posts.csv	Raw collected posts from Reddit
problematic_users.csv	Posts flagged by sentiment analysis
user_data_last_2_months.csv	Recent activity (posts/comments) per user
enriched_data.json	JSON version of enriched user activity
llm_analysis_results.csv	Final scores and explanations from Gemini LLM

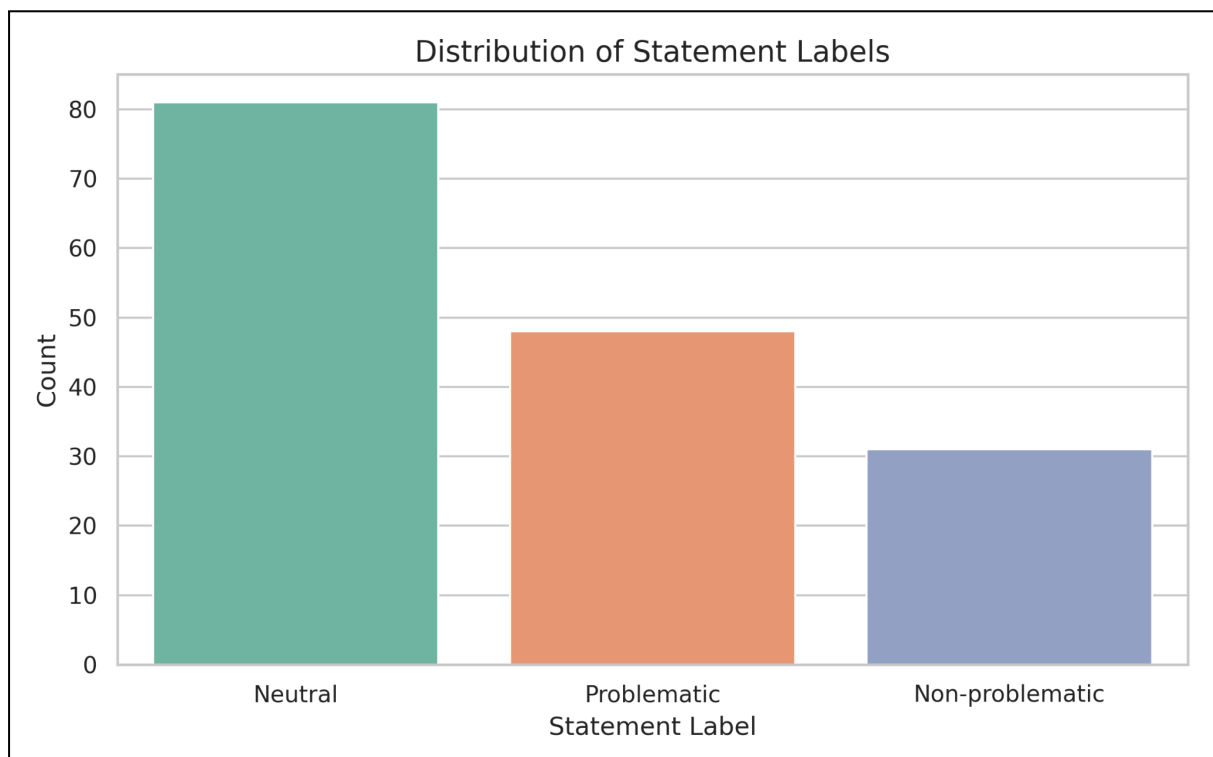
This structured approach ensures that every step of the pipeline can be reviewed, audited, or rerun with minimal manual effort.

3. Results

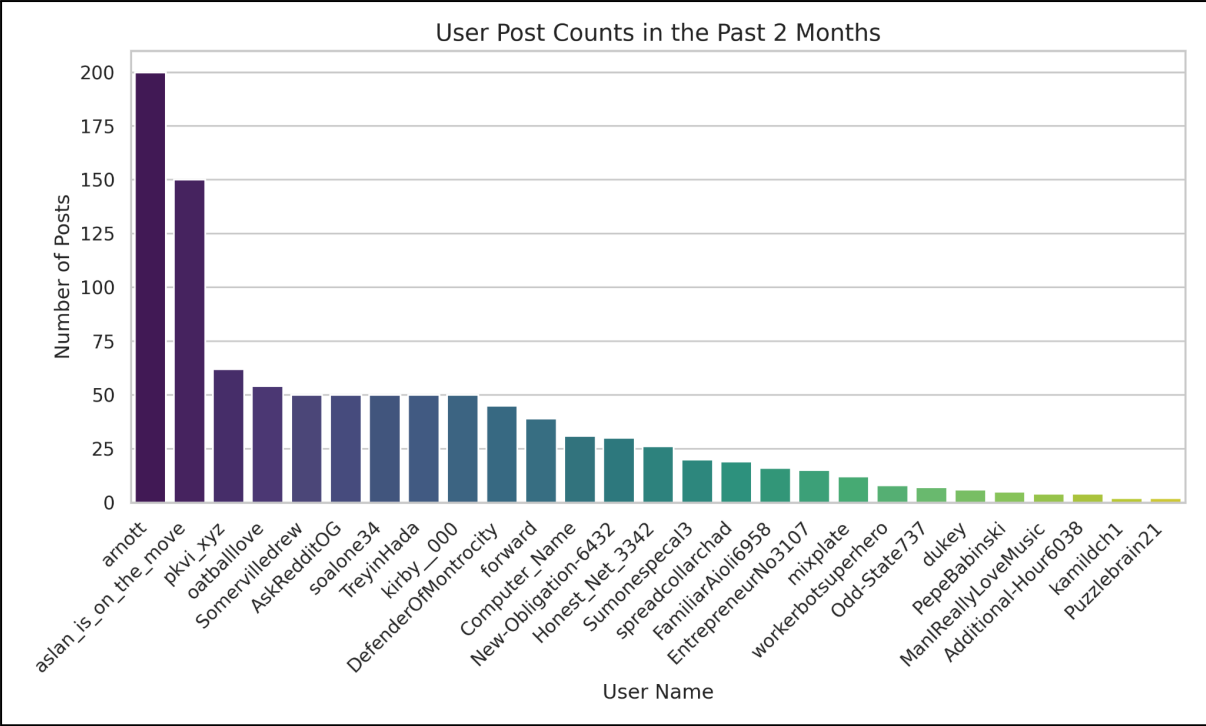
After running the full pipeline, starting from data collection to LLM analysis, we gathered meaningful insights into the types of Reddit content and users potentially involved in spreading antisemitic ideas. The system performed well across all stages, with consistent handling of edge cases and solid output generation.

3.1 Data Overview

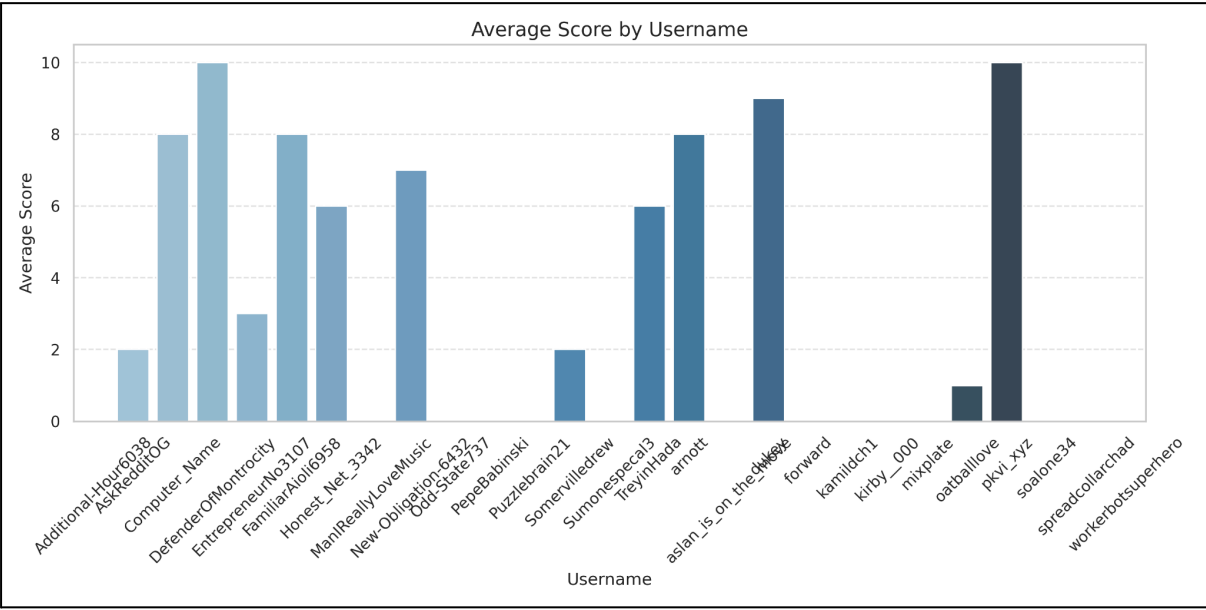
- **Posts Collected:** 160 posts from targeted subreddits matching sensitive or problematic keywords
- **Flagged as Problematic:** 48 posts, identified through sentiment analysis using NLTK's VADER tool
- **Unique Users Flagged:** 27 Reddit users associated with the problematic posts
- **User Data Collected:**
 - **2,431 items** (including posts, comments, or error logs)
 - Spanning **60 days** of user history per flagged account
- **Final Users Analyzed by LLM:** 27 users with sufficient data for scoring



Above figure shows the distribution of Reddit statements labeled as Neutral, Problematic, or Non-problematic.



Above figure illustrates the number of posts, user has posted over the last two months.



Furthermore, above figure displays the average problematic content score for each Reddit user analyzed

3.2 LLM-Based Scoring Outcomes

Using Gemini 2.5, each of the 27 users was evaluated based on their recent activity:

- The model provided both a **numerical radicalisation score** and a **text explanation**.

- These results were saved in `llm_analysis_results.csv` for further inspection or reporting.
- The explanations provided valuable context, showing whether users were engaging in clear antisemitic rhetoric or borderline behaviour.

3.3 Observations & Insights

- A significant portion of problematic content appeared in a handful of subreddits, suggesting possible echo chambers.
- The LLM was especially helpful in borderline cases where simple sentiment or keyword detection would have failed.
- A few flagged users had mostly neutral content but were part of threads or contexts where antisemitism was present indicating the value of context-aware scoring.
- Edge cases (e.g., deleted or private users) were handled gracefully, and logs were saved for those attempts.

4. Discussion

This project successfully demonstrates a working end-to-end system for identifying antisemitic behaviour on Reddit, from data collection and sentiment analysis to user behaviour scoring using a large language model (LLM). One of the biggest achievements was integrating an LLM-based evaluation pipeline, which significantly improves the system's intelligence and makes it feel much closer to a production-ready solution.

While many powerful LLMs are available today from providers like OpenAI, Anthropic (Claude), and various open-source LLM local models, they often come with challenges whether it's API costs, rate limits, or hardware requirements for local deployments.

To address this, I explored alternative solutions and discovered that Google AI Studio offers free access to the Gemini API, a robust and capable LLM with certain usage restrictions. After some experimentation and deeper research, I managed to work around many of these limitations (within acceptable usage policies), enabling live integration with Gemini 2.5 at no additional cost. While this required slightly more processing time and effort to manage request flow (like rate limiting), it was not a blocker.

The core pipeline remains flexible and modular. Its just that swapping in a new LLM would require minimal changes to the architecture.

4.1 Conclusion

This project proves that it's entirely possible to build a functional, intelligent content moderation and user-scoring system using freely available tools, open-source libraries, and cloud-based AI services using python as a programming language. The combination of Reddit data scraping, sentiment filtering, enrichment, and LLM-based analysis produced valuable insights and flagged users with context-aware justifications.

4.2 Future Scope

Even if the limitations that come with the free version of Gemini LLM API were more restrictive, they wouldn't pose a long-term problem. In future versions, the LLM layer could easily be replaced with:

- Paid APIs from providers like OpenAI or Claude for faster, higher-throughput requests.
- Locally hosted models for organisations prioritising data privacy or self-contained infrastructure.